

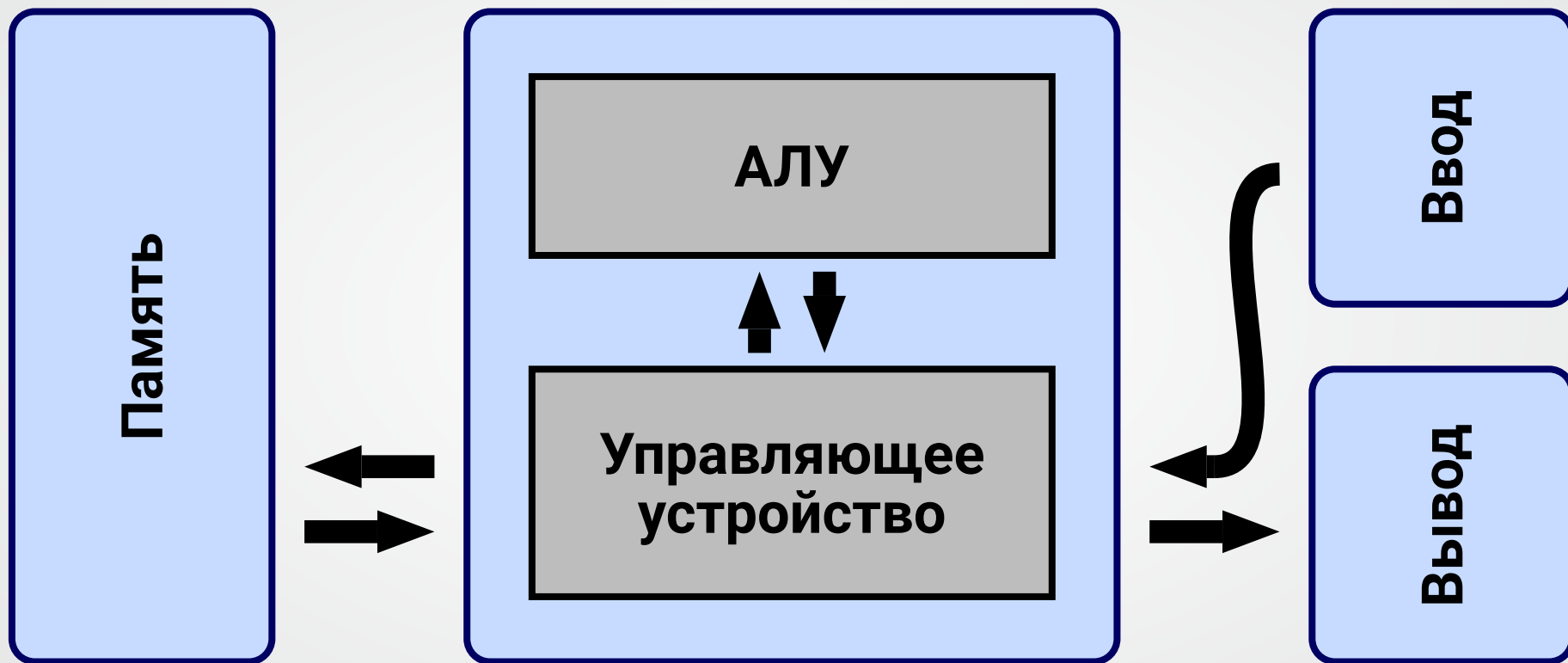
Ассемблер (x86)

АКОС, МФТИ

02 октября, 2025



Архитектура фон Неймана





Игра про создание своего процессора из реле

С чем работает процессор?

- + С тривиальными инструкциями, закодированными в бинарном виде;
- + С 8-, 16-, 32-, 64-битными числами (иногда до 512 бит);
- + С адресами памяти (опять же, в виде чисел).
- + Со стеком (регистр для указателя на стек `sp`, инструкции `push` / `pop`).

С чем не работает процессор?

- Структуры, классы, строки – всё это абстракции над адресами памяти;
- Массивы данных. Каждый элемент нужно обрабатывать отдельно.
- Функции, методы, исключения. Всё, что есть – это стек;

Задача процессора – уметь очень быстро выполнять простые инструкции, из которых уже можно составлять сложные программы.

Что такое инструкция?

mov **rbx**, **rax**

The diagram shows the instruction 'mov rbx, rax' with three horizontal brackets above each part. From 'mov', a vertical line descends to a horizontal line that branches to the label 'Мнемоника (название операции)'. From 'rbx', a vertical line descends to a horizontal line that branches to the label 'Первый операнд'. From 'rax', a vertical line descends to a horizontal line that branches to the label 'Второй операнд'.

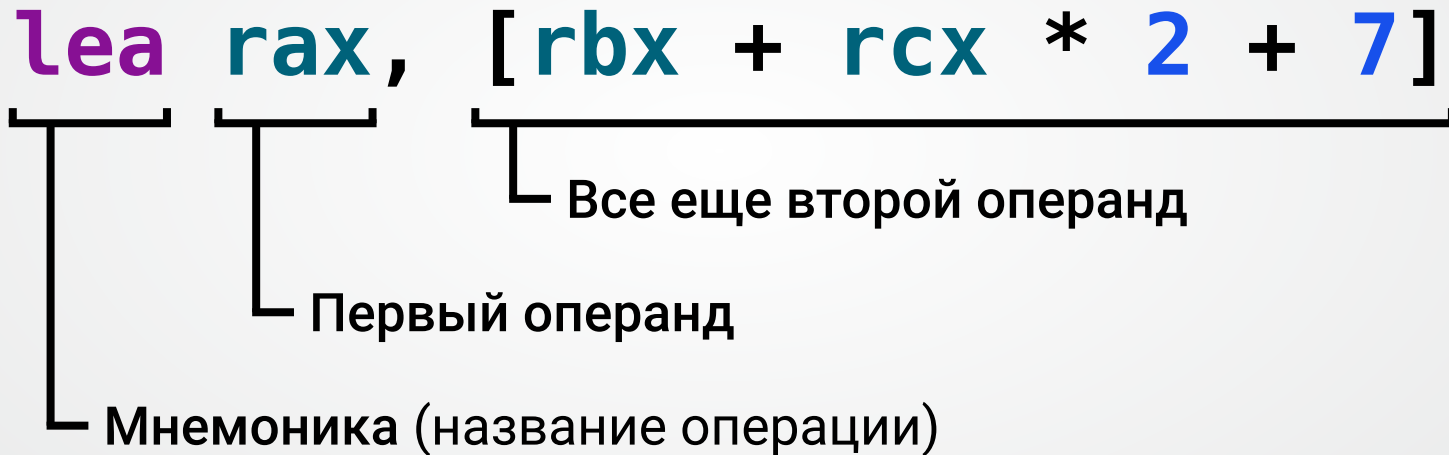
Мнемоника (название операции)

Первый операнд

Второй операнд

Для процессора она выглядит как **48 89 C3**

Что такое инструкция?



Для процессора она выглядит как **48 8D 44 4B 07**

Наборы инструкций

x86 (x86_64)

- Наиболее распространённый на **настольных компьютерах** / ноутбуках;
- Использует **CISC** - Complex Instruction Set.

arm(v1 ... v8)

- Живёт на **большинстве телефонов**, роутеров, во встраиваемых системах;
- Сейчас захватывает и настольный рынок (Apple M1, Snapdragon X);
- Использует **RISC** - Reduced Instruction Set.

Еще есть MIPS и PowerPC , но их мы не затронем.

CISC : инструкции могут иметь разную длину

`nop`

`0x90`

`mov rbx, rax`

`0x48 0x89 0xC3`

`cmovg rax, r15`

`0x49 0x0F 0x4F 0xC7`

RISC : все инструкции равнодлинные

`nop`

`0x1F 0x20 0x03 0xD5`

`mov x1, x0`

`0xE0 0x03 0x01 0xAA`

`cse1 x0, x15, x0, gt`

`0xE0 0xC1 0x80 0x9A`

x86

16-битные регистры общего назначения

ax	Accumulator
bx	Base index (для работы с массивами)
cx	Counter
dx	Accumulator extension
r8w - r15w	У этих даже названия нет, просто регистры
sp	Stack pointer
bp	Base pointer
si	Source index
di	Destination index

Специальные 16-битные регистры:

ip	Instruction pointer
flags	Скрытый регистр флагов: переполнение, чётность, знак, ...

Вложенность регистров – **ax**

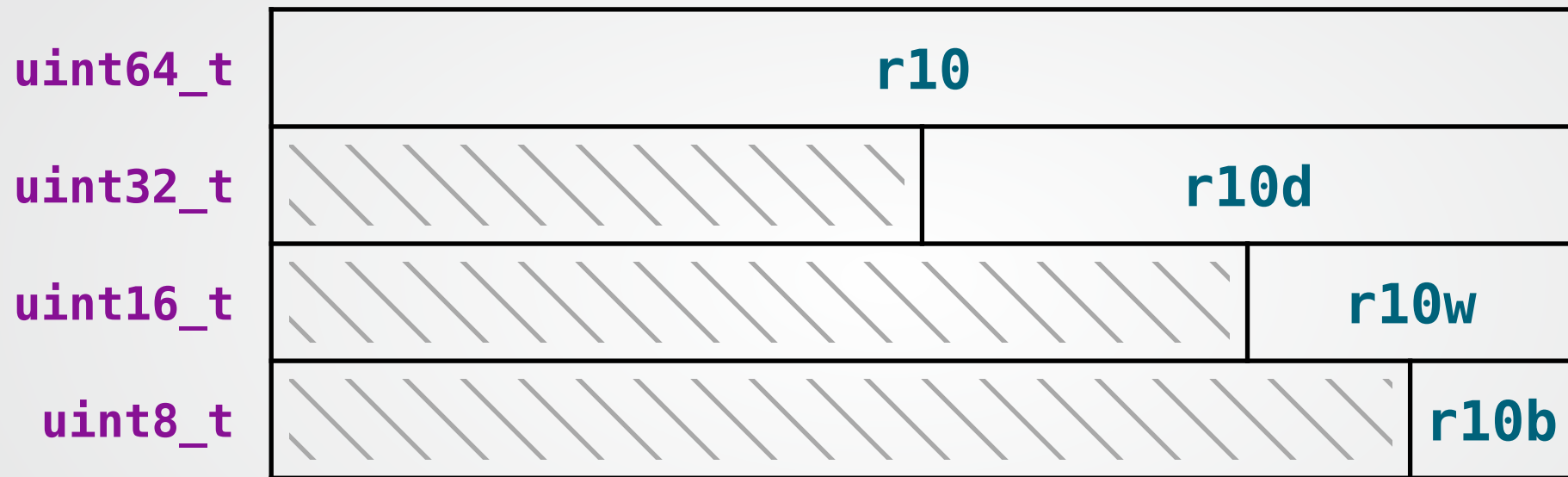
uint64_t	rax		
uint32_t			eax
uint16_t			ax
uint8_t			ah
			al

Аналогично работают **bx**, **cx**, **dx**

sp, **bp**, **si**, и **di**, не имеют вариантов с **h** в конце.

ip в чистом виде не встречается, только **eip** и **rip**.

Вложенность регистров – r10



Аналогично работают r8 ... r15

Intel

```
mov rax, 5  
lea eax, [ecx + ebx * 2 + 7]  
and DWORD PTR [eax], 7
```

- Порядок операндов: куда ← откуда
- Адреса: `[base + index * scale + offset]`
- Битность инструкции обычно угадывается;
- Если не угадывается, то есть директивы:
 - `BYTE PTR` - 1 байт
 - `WORD PTR` - 2 байта
 - `DWORD PTR` - 4 байт
 - `QWORD PTR` - 8 байт

AT&T

```
movq $5, %rbx  
leal 7(%ecx, %ebx, 2), %eax  
andl $7, (%eax)
```

- Порядок операндов: откуда → куда
- Адреса: `offset(base, index, scale)`
- Битность явно указывается суффиксом:
 - `b` - 1 байт
 - `w` - 2 байта
 - `l` - 4 байт
 - `q` - 8 байт
- Перед каждым регистром %, перед каждым числом \$

Базовые инструкции

<code>mov</code>	Перенести значение между регистрами / памятью.
<code>lea</code>	Загрузить адрес второго операнда в первый.
<code>add</code> <code>sub</code> <code>and</code> ...	Арифметика / логика над значением в регистре / памяти. Работают как <code>+=</code> , <code>-=</code> , <code>&=</code> , <code> =</code> , <code>^=</code> , <code>~=</code> , ... в Си.
<code>neg</code>	Поменять знак значения в регистре / памяти
<code>imul</code> <code>mul</code>	Знаковое / беззнаковое умножение.
<code>idiv</code> <code>div</code>	Знаковое / беззнаковое деление.
<code>cmp</code>	Математическое сравнение величин (результат в <code>flags</code>).
<code>test</code>	Побитовое сравнение величин (результат в <code>flags</code>).
<code>push</code> <code>pop</code>	Добавить / считать значение со стека.
<code>jmp</code>	Переход в другое место программы.
<code>syscall</code>	Системный вызов.

Минимальная корректная программа на ассемблере:

```
1  _main:  
2      mov rax, SYS_EXIT  
3      mov rdi, (код возврата)  
4      syscall
```

- Выполняет **системный вызов** `exit(int retval)`
- **Номер** системного вызова передаётся в `rax`
- **Аргумент** системного вызова передаётся в `rdi`
- **Системный вызов** выполняется инструкцией `syscall`.

Как выразить вызов функции?


```
1  uint64_t global_var;
2
3  void my_func() {
4      // Делаем что-то полезное здесь
5      global_var = 42;
6  }
7
8  void main() {
9      my_func();
10     __exit(global_var);
11 }
```

```
1  # global_var = r10
2
3  _my_func:
4      mov r10, 42
5      # return ...?
6
7  _main:
8      # _my_func() ...?
9
10     mov rax, SYS_EXIT
11     mov rdi, r10
12     syscall
```

Вариант 1

Вызов функции

mov r10, rip

add r10, 11

jmp _my_func

Выход из функции

jmp r10

* Сдвиг не всегда будет 11 байт, размер инструкции **jmp** может отличаться

Вариант 1

А так нельзя!

Вызов функции

mov r10, rip

add r10, 11

jmp _my_func

Выход из функции

jmp r10



* Сдвиг не всегда будет 11 байт, размер инструкции **jmp** может отличаться

Вариант 2 – `lea`

Вызов функции

```
lea r10, [rip + 5]
```

```
jmp _my_func
```

Выход из функции

```
jmp r10
```

* И здесь, сдвиг не всегда будет 5 байт

Вариант 2 – `lea`

Вызов функции

```
lea r10, [rip + 5]
```

```
jmp _my_func
```

Выход из функции

```
jmp r10
```

Так можно,
но закончатся регистры



* И здесь, сдвиг не всегда будет 5 байт

Вариант 3 – `lea + push`

Вызов функции

```
lea rax, [rip + 6]
```

```
push rax
```

```
jmp _my_func
```

Выход из функции

```
pop rax
```

```
jmp rax
```

* Здесь тоже сдвиг может быть другой

Вариант 4 – а что, так можно было?

Вызов функции
`call _my_func`

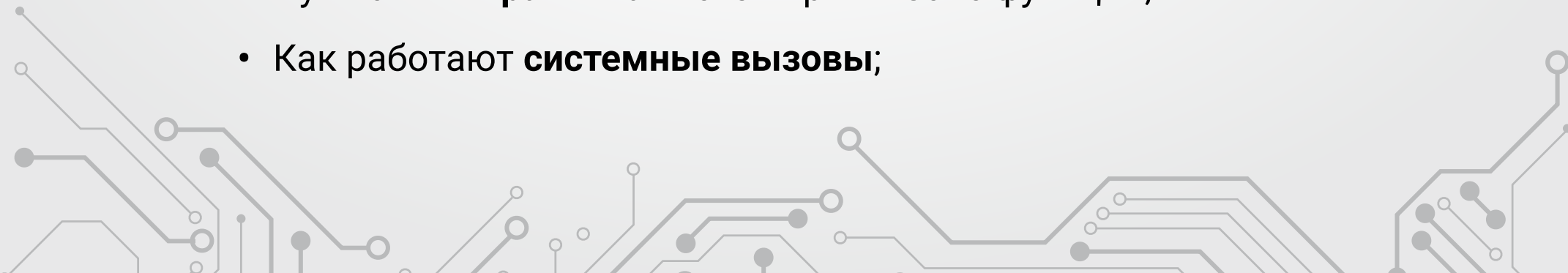
Выход из функции
`ret`

Соглашение о вызове

(Calling convention)

Что включает в себя соглашение о вызове?

- Где хранится **адрес возврата**;
- Как передаются **аргументы** и возвращаемое значение;
- Какие регистры можно **перезаписывать**, а какие надо **восстанавливать** перед выходом из функции;
- Нужно ли **выравнивать стек** при вызове функции;
- Как работают **системные вызовы**;



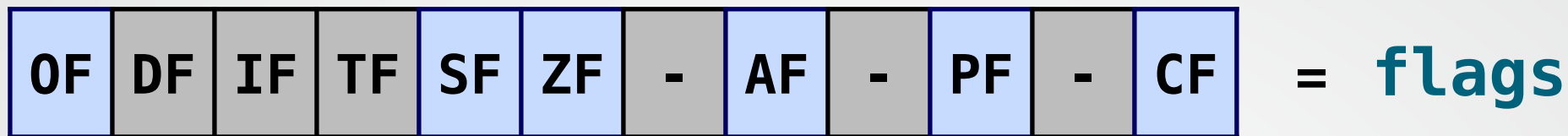
System V

Calling Convention

- Аргументы передаются через **rdi, rsi, rdx, rcx, r8, r9** ;
- **float** и **double** - передаются через **xmm0 - xmm7** ;
- То, что не влезло, передаётся **через стек** – справа налево;
- Возвращаемое значение передается в **rax** (и в **rdx**, если не влезает);
- **float** и **double** - возвращаются через **xmm0 - xmm1** ;
- **Стек выравнивается** по 16 байт перед вызовом;
- **rbx, rsp, rbp, r12, r13, r14, r15** нужно восстанавливать перед возвратом;
- Номер системного вызова передается через **rax**.

Сеанс магии

Регистр флагов



Carry Flag – беззнаковое переполнение
Parity Flag – последний результат был чётным
Auxiliary Carry Flag – переполнение нижнего слова
Zero Flag – последний результат равен нулю
Sign Flag – последний результат был чётным
Trap Flag – системный флаг
Interrupt Enable Flag – системный флаг
Direction Flag – управляющий флаг
Overflow Flag – знаковое переполнение

Но как им пользоваться, если он скрыт?

Условные переносы - **cmov****

cmovs | Перенести, если установлен **Sign Flag**

cmovz | Перенести, если установлен **Zero Flag**

cmove |

cmovp | Перенести, если установлен **Parity Flag**

cmovpe |

cmovns | То же самое, но наоборот (если флаг **не** установлен)

cmovnz |

cmovne |

cmovnp |

cmovpo |

Условные переходы - j**

js		Перейти, если установлен Sign Flag
jz, je		Перейти, если установлен Zero Flag
jp, jpo		Перейти, если установлен Parity Flag

Ещё есть **cset**** - установить один байт в **0** или **1** в зависимости от флагов.

Больше условных переходов!

Представим, что мы вычли два числа: `sub rax, rbx`. Тогда:

- `rax >= rbx`, если `CF = 0`
- `rax <= rbx`, если `CF = 1`
- `rax == rbx`, если `ZF = 0`

А если числа были **знаковые**, то:

- `rax >= rbx`, если `SF = 0F`
- `rax <= rbx`, если `SF != 0F`

Для этого придумали свои суффиксы:

<code>jge</code>	Перейти, если знаково больше или равно (<code>SF = 0F</code>)
<code>ja</code>	Перейти, если беззнаково больше (<code>CF = 1</code> и <code>ZF = 0</code>)
<code>jl</code>	Перейти, если знаково меньше (<code>SF != 0F</code> и <code>ZF = 0</code>)

Но `sub` поменяет значение `rax`. `cmp` тоже вычитает, но оставляет все как есть.

Интерактив

Стековый фрейм

```
1  void merge_sort(int* arr, int n) {  
2      // Где хранить локальные переменные?  
3      int mid = n / 2;  
4      int left_size = mid, right_size = n - mid;  
5  
6      // ...  
7      merge_sort(left, left_size);  
8      merge_sort(right, right_size);  
9      // ...  
10 }
```

Стековый фрейм

1 `_merge_sort:`

2 `push rbp`

3 `mov rbp, rsp`

4 `sub rsp, 16` # 16 байт для фрейма

5 `# ...`

6 `mov [rbp - 4], rax` # int mid = ...

7 `mov [rbp - 8], rax` # int left_size = ...

8 `mov [rbp - 12], rax` # int right_size = ...

9 `# ...`

10 `mov rsp, rbp`

11 `pop rbp`

12 `ret`

Создание фрейма

Очистка фрейма

Стековый фрейм

- Если начинать каждую функцию с `push rbp` и `mov rbp, rsp`, то фреймы образуют односвязный список.
- Итерируя по нему, дебаггеры могут показать бектрейс:

```
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = instruction step into
  * frame #0: 0x0000000100003f00 a.out`fib
    frame #1: 0x0000000100003f2d a.out`fib_body + 19
    frame #2: 0x0000000100003f23 a.out`fib_body + 9
    frame #3: 0x0000000100003f23 a.out`fib_body + 9
    frame #4: 0x0000000100003f23 a.out`fib_body + 9
    frame #5: 0x0000000100003f23 a.out`fib_body + 9
    frame #6: 0x0000000100003f23 a.out`fib_body + 9
    frame #7: 0x0000000100003f23 a.out`fib_body + 9
    frame #8: 0x0000000100003f23 a.out`fib_body + 9
    frame #9: 0x0000000100003f23 a.out`fib_body + 9
    frame #10: 0x0000000100003f42 a.out`asm_func + 12
    frame #11: 0x0000000100003f8b a.out`main + 11
    frame #12: 0x0000000200012310 dyld`start + 2432
```

Полезные ссылки

-  godbolt.org – Онлайн-компилятор C / C++ в ассемблер;
-  [online-x86-assembler](https://online-x86-assembler.com) – Онлайн-компилятор ассемблера в машинный код x86;
-  nandgame.com – Игра про создание процессора из реле. Советую!



Спасибо за внимание!



github.com/JakMobius/courses/tree/mipt-os-basic-2025/